Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Отчёт
По лабораторной работе № 3
по дисциплине Технологии Виртуализации
Вариант: -

Работу выполнил:
Молчанов Федор Денисович P3413

Работу принял:
Белозубов А.В.

г. Санкт-Петербург 2026

Оглавление

Цель работы 2
Ход работы 2
Пункт 1. Проверка Docker и VirtualBox 2
Пункт 2. Запуск Hello-Молчанов 2
Пункт 3. Nginx и HTML-страница 2
Пункт 4. MariaDB 3
Пункт 5. Nextcloud 3
Пункт 6. phpVirtualBox 4
Пункт 7. Проект Docker Compose 5
Пункт 8. Регистрация на Docker Hub 7
Пункт 9. Публикация образа 7
Пункт 10. Запуск игры из Docker Hub 7
Пункт 11. Основные команды 8
Пункт 12. Portainer 8
Вывод 9

Цель работы

Изучить создание и запуск контейнеров Docker, подключение каталогов хоста, публикацию портов и объединение сервисов с помощью Docker Compose. В качестве веб-сервера используется Nginx.

Ход работы

Пункт 1. Проверка Docker и VirtualBox

Работа выполнена в Arch Linux. Docker и VirtualBox были установлены ранее, поэтому повторная установка не потребовалась.

```
$ docker --version
Docker version 29.8.2, build 7fc2dff9bc
$ docker compose version
Docker Compose version 5.6.0
```

Модуль VirtualBox загружен в терминале хоста:

```
$ sudo modprobe vboxdrv
$ VBoxManage --version
7.2.20r175154
$ ls -l /dev/vboxdrv
crw------- 1 root root 10, 261 Oct 21 16:39 /dev/vboxdrv
$ VBoxManage list vms
"WS_MolchanovFyodorDenisovich_ubuntu" {c36d64f2-beb3-44cc-9263-a58a85fa6e6d}
"ubuntu-22" {f5a5a4d5-5149-4d14-8b19-d0ceb348ca26}
"ensp" {513dfaa6-b3d8-4286-83da-a574fb64a267}
```

Модуль загружен, предупреждение отсутствует, список зарегистрированных виртуальных машин получен.

Пункт 2. Запуск Hello-Молчанов

Образу `hello-world` присвоен локальный тег `hello-molchanov`. Имя записано латиницей в нижнем регистре согласно формату имён образов Docker. Все действия выполнены в терминале. Ниже приведён основной фрагмент вывода.

```
$ docker pull hello-world:latest
Status: Downloaded newer image for hello-world:latest
$ docker tag hello-world:latest hello-molchanov:latest
$ docker run --name lab3-hello hello-molchanov
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

Контейнер выполнил программу и завершился. Успешный вывод подтверждает работу клиента, Docker Engine и запуска контейнеров.

Пункт 3. Nginx и HTML-страница

Создан образ `lab3-nginx:1.0`. Файл `nginx/Dockerfile`:

```
FROM nginx:stable-alpine
COPY default.conf /etc/nginx/conf.d/default.conf
EXPOSE 8080
```

В `nginx/default.conf` задан порт 8080:

```
server {
    listen 8080;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;
}
```

Страница хранится только на хосте в `/home/www/html/index.html`. Каталог подключается к контейнеру для чтения. Dockerfile задаёт конфигурацию Nginx; HTML в образ не копируется. Команда запуска Nginx наследуется от базового образа.

```
$ cat /home/www/html/index.html
<!doctype html>
<html lang="ru">
<meta charset="utf-8">
<title>Лабораторная работа № 3</title>
<h1>Молчанов Фёдор Денисович</h1>
<p>Группа P3413. Лабораторная работа № 3. Nginx в Docker.</p>
</html>
```

```
$ docker build -t lab3-nginx:1.0 nginx
Successfully tagged lab3-nginx:1.0
$ docker run -d --name lab3-nginx \
    -p 127.0.0.1:8080:8080 \
    -v /home/www/html:/usr/share/nginx/html:ro lab3-nginx:1.0
$ curl -i http://127.0.0.1:8080/
HTTP/1.1 200 OK
Server: nginx/1.30.6
Content-Type: text/html
```

Тело ответа содержит созданную HTML-страницу. Сервис доступен по адресу http://localhost:8080. Порты лабораторных сервисов опубликованы на loopback-интерфейсе хоста.

Пункт 4. MariaDB

Запущен официальный образ `mariadb:11.4`. Данные сохраняются в каталоге хоста `/home/DB`, подключённом к `/var/lib/mysql`. Опция `-w` также задаёт рабочий каталог контейнера `/home/DB`. Использован стандартный порт MariaDB 3306. Пароли передаются из локального файла `.env` и в отчёт не включены.

```
$ docker pull mariadb:11.4
Status: Downloaded newer image for mariadb:11.4
$ docker run -d --name lab3-mariadb --env-file .env \
    -p 127.0.0.1:3306:3306 -v /home/DB:/var/lib/mysql \
    -w /home/DB mariadb:11.4
$ docker exec lab3-mariadb sh -c \
    'MYSQL_PWD="$MARIADB_ROOT_PASSWORD" mariadb -uroot -e \
    "CREATE DATABASE IF NOT EXISTS molchanov CHARACTER SET utf8mb4; \
    SHOW DATABASES;"'
Database
information_schema
molchanov
mysql
performance_schema
sys
```

Подключение к СУБД выполнено через клиент `mariadb` в контейнере. Создана база `molchanov`, названная по фамилии студента.

Пункт 5. Nextcloud

Загружен официальный образ `nextcloud:33-fpm`. Приложение PHP-FPM принимает FastCGI-запросы на порту 9000; HTTP-запросы обслуживает отдельный контейнер Nginx. Каталог хоста `/home/www/NextCloud` подключён к обоим контейнерам, для Nginx — только для чтения. Для отдельного учебного экземпляра использована SQLite; связка с MariaDB реализована в пункте 7.

```
$ docker pull nextcloud:33-fpm
Status: Downloaded newer image for nextcloud:33-fpm
$ docker network create lab3-standalone
$ docker run -d --name lab3-nextcloud \
```

```
    --network lab3-standalone --network-alias app --env-file .env \
    -e NEXTCLOUD_TRUSTED_DOMAINS='localhost 127.0.0.1' \
    -v /home/www/NextCloud:/var/www/html nextcloud:33-fpm
$ docker build -t lab3-nextcloud-nginx:1.0 compose
$ docker run -d --name lab3-nextcloud-web \
    --network lab3-standalone -p 127.0.0.1:8088:80 \
    -v /home/www/NextCloud:/var/www/html:ro lab3-nextcloud-nginx:1.0
```

Образ Nginx собран с конфигурацией из официального примера Nextcloud. Полная конфигурация сохранена в `compose/nginx.conf`; она задаёт корень `/var/www/html`, обработку статических файлов и передачу PHP-запросов на `app:9000`. Файл `compose/Dockerfile`:

```
FROM nginx:stable-alpine
COPY nginx.conf /etc/nginx/nginx.conf
```

Установка выполнена через консольную утилиту `occ` от имени `www-data`. Перед установкой исправлен владелец каталога конфигурации.

```
$ docker exec lab3-nextcloud chown -R www-data:www-data /var/www/html/config
$ docker exec -u www-data lab3-nextcloud sh -c \
    'php occ maintenance:install --database=sqlite \
    --admin-user="$NEXTCLOUD_ADMIN_USER" \
    --admin-pass="$NEXTCLOUD_ADMIN_PASSWORD"'
Nextcloud was successfully installed
$ docker exec -u www-data lab3-nextcloud \
    php occ config:system:set trusted_domains 1 --value=127.0.0.1
$ docker exec -u www-data lab3-nextcloud \
    php occ user:setting molchanov settings display_name \
    'Молчанов Фёдор Денисович'
$ docker exec -u www-data lab3-nextcloud sh -c \
    'php occ user:add --password-from-env \
    --display-name="Белозубов А. В." belozubov'
$ docker exec -u www-data lab3-nextcloud \
    php occ user:setting belozubov settings display_name \
    'Белозубов Александр Владимирович'
$ docker exec -u www-data lab3-nextcloud php occ user:list
  - belozubov: Белозубов Александр Владимирович
  - molchanov: Молчанов Фёдор Денисович
```

Созданы администратор `molchanov` и обычный пользователь `belozubov`. Адрес приложения: http://localhost:8088.

Пункт 6. phpVirtualBox

Загружен и запущен образ `joweisberg/phpvirtualbox:latest`. Согласно документации образа, интерфейс управляет VirtualBox через сервис хоста `vboxwebsrv`. Для обращения к хосту добавлено имя `host.docker.internal`.

```
$ docker pull joweisberg/phpvirtualbox:latest
Status: Downloaded newer image for joweisberg/phpvirtualbox:latest
$ docker run -d --name lab3-phpvirtualbox -p 127.0.0.1:8090:80 \
    --add-host host.docker.internal:host-gateway \
    -e VBOX_HOSTPORT=host.docker.internal:18083 \
    -e VBOX_NAME=archbase joweisberg/phpvirtualbox:latest
$ curl -fsS -o /tmp/lab3-phpvirtualbox.html \
    -w 'HTTP %{http_code}\n' http://127.0.0.1:8090/
HTTP 200
$ rg -o '<title>[^<]+' /tmp/lab3-phpvirtualbox.html
<title>phpVirtualBox - VirtualBox Web Console
```

Веб-интерфейс доступен по адресу http://localhost:8090. После загрузки модуля VirtualBox пользователь запустил веб-сервис на хосте от имени `theodor`. Адрес `172.17.0.1` — шлюз сети Docker:

```
$ vboxwebsrv -H 172.17.0.1 -p 18083
Oracle VirtualBox web service Version 7.2.20
Socket connection successful: host = 172.17.0.1, port = 18083, master socket = 9
```

В конфигурации phpVirtualBox указаны учётные данные пользователя хоста `theodor`. Файл конфигурации доступен для чтения пользователю PHP-FPM:

```
$ docker exec lab3-phpvirtualbox chown nobody:nobody /var/www/config-servers.php
```

Пользователь выполнил вход в веб-интерфейс с начальной учётной записью `admin`. Отображаются VirtualBox версии 7.2.20 и все три зарегистрированные машины. Соединение с гипервизором и получение списка машин подтверждены. Запуск виртуальных машин в рамках проверки не выполнялся.

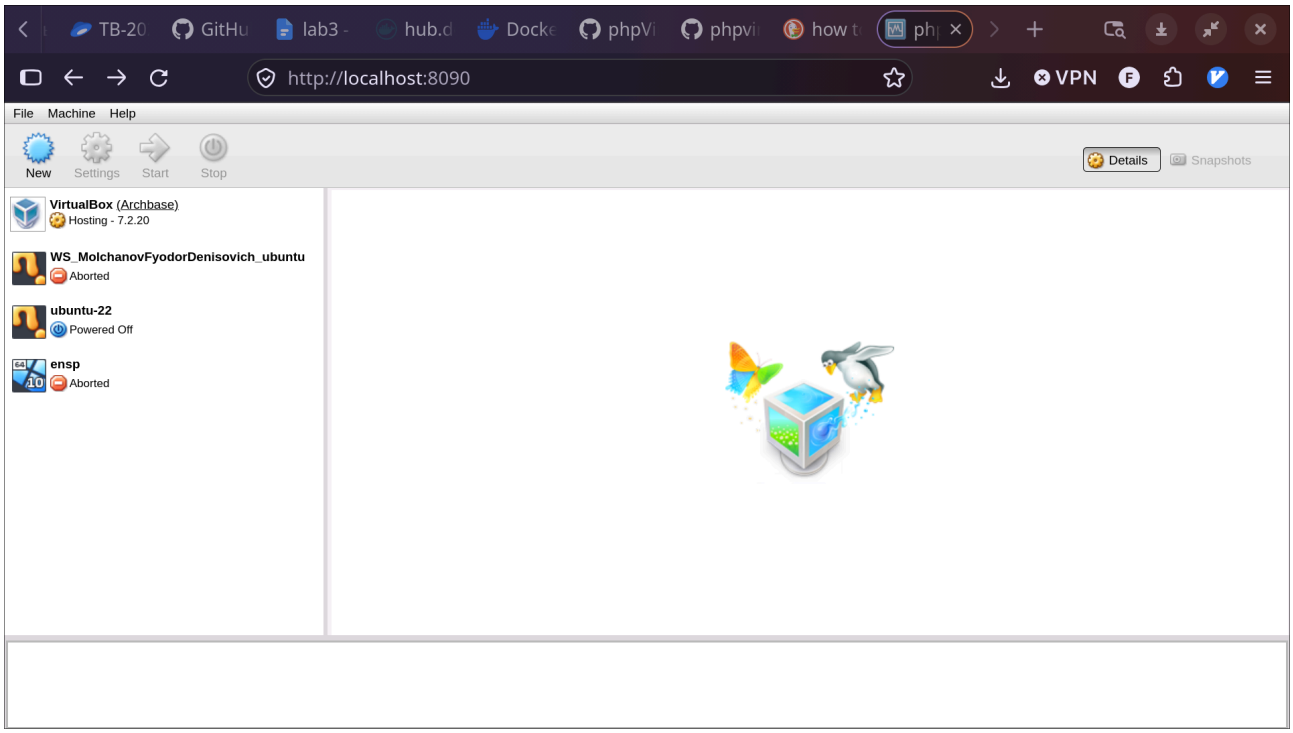


Рис. 1. phpVirtualBox: подключение к хосту Archbase и список виртуальных машин.

Пункт 7. Проект Docker Compose

Создан отдельный проект `lab3-cloud` из трёх сервисов: `web` (Nginx), `app` (Nextcloud PHP-FPM) и `db` (MariaDB). Запросы поступают на порт 2022 хоста, затем Nginx передаёт PHP-запросы в `app:9000`, а Nextcloud обращается к MariaDB по имени `db` на порту 3306. Имена разрешаются во внутренней сети Compose. Порт БД этого проекта на хост не опубликован.

Файл `compose.yaml` в итоговой редакции (имя образа изменено в пункте 9, сервис Portainer добавлен в пункте 12):

```yaml
name: lab3-cloud
services:
  db:
    image: mariadb:11.4
    restart: unless-stopped
    environment:
      MARIADB_ROOT_PASSWORD: ${MARIADB_ROOT_PASSWORD:?}
      MARIADB_DATABASE: nextcloud
      MARIADB_USER: nextcloud
      MARIADB_PASSWORD: ${MARIADB_PASSWORD:?}
    volumes:
```

```
      - db_data:/var/lib/mysql
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 5s
      timeout: 5s
      retries: 30
  app:
    image: nextcloud:33-fpm
    restart: unless-stopped
    depends_on:
      db:
        condition: service_healthy
    environment:
      MYSQL_HOST: db
      MYSQL_DATABASE: nextcloud
      MYSQL_USER: nextcloud
      MYSQL_PASSWORD: ${MARIADB_PASSWORD:?}
      NEXTCLOUD_ADMIN_USER: ${NEXTCLOUD_ADMIN_USER:?}
      NEXTCLOUD_ADMIN_PASSWORD: ${NEXTCLOUD_ADMIN_PASSWORD:?}
      NEXTCLOUD_TRUSTED_DOMAINS: localhost 127.0.0.1
      OVERWRITECLIURL: http://localhost:2022
    volumes:
      - nextcloud_data:/var/www/html
  web:
    build: ./compose
    image: fefumo/lab3-nextcloud-nginx:1.1
    restart: unless-stopped
    depends_on:
      - app
    ports:
      - "127.0.0.1:2022:80"
    volumes:
      - nextcloud_data:/var/www/html:ro
  portainer:
    image: portainer/portainer-ce:lts
    restart: unless-stopped
    ports:
      - "127.0.0.1:9443:9443"
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - portainer_data:/data
volumes:
  db_data:
  nextcloud_data:
  portainer_data:
```

Том `nextcloud_data` содержит файлы приложения и пользователей, `db_data` — данные MariaDB. Проверка `healthcheck` обеспечивает готовность БД до запуска приложения. Значения переменных подставляются из `.env`; файл `.env.example` содержит шаблон без действующих паролей.

```
$ docker compose up -d
 Container lab3-cloud-db-1 Started
 Container lab3-cloud-db-1 Healthy
 Container lab3-cloud-app-1 Started
 Container lab3-cloud-web-1 Started
$ docker compose exec -T web nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
$ docker compose exec -T -u www-data app php occ config:system:get dbtype
mysql
```

```
$ docker compose exec -T -u www-data app php occ config:system:get dbhost
db
```

Значение `mysql` обозначает используемый Nextcloud драйвер подключения к MariaDB. В отличие от отдельного экземпляра пункта 5, здесь используется серверная СУБД. Установка выполнена автоматически через переменные окружения. Отображаемое имя администратора задано командой `occ user:setting`, как в пункте 5. Проверка доступа к приложению:

```
$ curl -fsS -o /dev/null -w 'Nextcloud login: HTTP %{http_code}\n' \
    http://127.0.0.1:2022/login
Nextcloud login: HTTP 200
```

Команда `docker compose ps` показала три работающих контейнера; для MariaDB установлено состояние `healthy`. Проверка `occ status` подтвердила `installed: true`, `maintenance: false` и `needsDbUpgrade: false`. Приложение доступно по адресу http://localhost:2022.

Пункт 8. Регистрация на Docker Hub

Создана учётная запись `fefumo` и репозиторий fefumo/lab3-nextcloud-nginx. Регистрация и создание репозитория выполнены пользователем через веб-интерфейс.

Пункт 9. Публикация образа

После авторизации командой `docker login` под учётной записью `fefumo` образ Nginx с конфигурацией Nextcloud собран командой Compose и опубликован. Ниже приведены основные строки вывода:

```
$ docker compose build web
$ docker push fefumo/lab3-nextcloud-nginx:1.1
1.1: digest: sha256:e1f1c34756896adea93121215db1b1555b85de7627dea4240e5de308cb2ea9e2 size: 2197
$ docker pull fefumo/lab3-nextcloud-nginx:1.1
Status: Image is up to date for fefumo/lab3-nextcloud-nginx:1.1
$ docker compose up -d --no-build web
 Container lab3-cloud-web-1 Recreated
 Container lab3-cloud-web-1 Started
$ curl -fsS -o /dev/null -w 'Nextcloud login: HTTP %{http_code}\n' \
    http://127.0.0.1:2022/login
Nextcloud login: HTTP 200
```

Образ опубликован с тегом `1.1`. Команда `pull` подтвердила доступность образа в реестре; слои уже присутствовали локально. Сервис `web` пересоздан с опубликованным именем образа без локальной сборки, Nextcloud отвечает HTTP 200. В `compose.yaml` указано имя `fefumo/lab3-nextcloud-nginx:1.1`. Опубликован образ веб-сервера; Nextcloud и MariaDB используют официальные образы, а файл Compose описывает их совместный запуск. Данные и пароли пользователей в образ не включаются.

Пункт 10. Запуск игры из Docker Hub

Загружен готовый образ `marcincuber/2048-game:latest-new` из Docker Hub. Контейнер публикует HTTP-порт приложения на порту 2048 хоста.

```
$ docker pull marcincuber/2048-game:latest-new
Status: Downloaded newer image for marcincuber/2048-game:latest-new
$ docker run -d --name lab3-2048 -p 127.0.0.1:2048:80 \
    marcincuber/2048-game:latest-new
$ curl -fsS -o /tmp/lab3-2048.html -w '2048: HTTP %{http_code}\n' \
    http://127.0.0.1:2048/
2048: HTTP 200
$ rg -o '<title>[^<]+' /tmp/lab3-2048.html
<title>2048
```

Игра доступна по адресу http://localhost:2048.

Пункт 11. Основные команды

Команда	Назначение
`docker pull`	Загрузка образа из реестра.
`docker build -t`	Сборка образа из Dockerfile с назначением тега.
`docker tag`, `docker push`	Назначение имени образа и публикация в реестре.
`docker run -d`	Создание и запуск контейнера в фоновом режиме.
`-p`, `-v`, `--env-file`	Публикация порта, подключение хранилища, передача переменных окружения.
`docker ps`, `docker logs`	Просмотр состояния контейнеров и журналов.
`docker exec`, `docker cp`	Выполнение команды в работающем контейнере и копирование файлов.
`docker inspect`	Просмотр параметров контейнера, включая подключённые каталоги.
`docker compose up -d`	Создание и запуск сервисов проекта.
`docker compose ps`, `exec`	Проверка сервисов и выполнение команд внутри них.

В Dockerfile инструкция `FROM` выбирает базовый образ, `WORKDIR` задаёт рабочий каталог, `COPY` добавляет файлы, `EXPOSE` указывает порт приложения, а `CMD` задаёт команду запуска. `EXPOSE` не публикует порт на хосте; для этого используется `-p` или `ports` в Compose.

Пункт 12. Portainer

В проект добавлен сервис `portainer` на основе образа `portainer/portainer-ce:lts` по официальной инструкции. Том `portainer_data` сохраняет настройки, подключение `/var/run/docker.sock` обеспечивает управление локальным Docker Engine. Интерфейс опубликован на порту 9443 только для локального доступа.

```
$ docker compose config -q
$ docker compose up -d portainer
 Container lab3-cloud-portainer-1 Created
 Container lab3-cloud-portainer-1 Started
$ curl -kfsS -o /dev/null -w 'Portainer: HTTP %{http_code}\n' \
    https://127.0.0.1:9443/
Portainer: HTTP 200
```

Адрес интерфейса: https://localhost:9443. Опция `-k` использована для проверки локального самоподписанного сертификата. Через веб-интерфейс создан администратор `admin`, выбрано локальное окружение `local` и открыт раздел `Containers`. В списке отображаются контейнеры лабораторной, их образы, состояния и опубликованные порты. Сервисы Nextcloud, Nginx и Portainer работают; MariaDB проекта имеет состояние `healthy`.

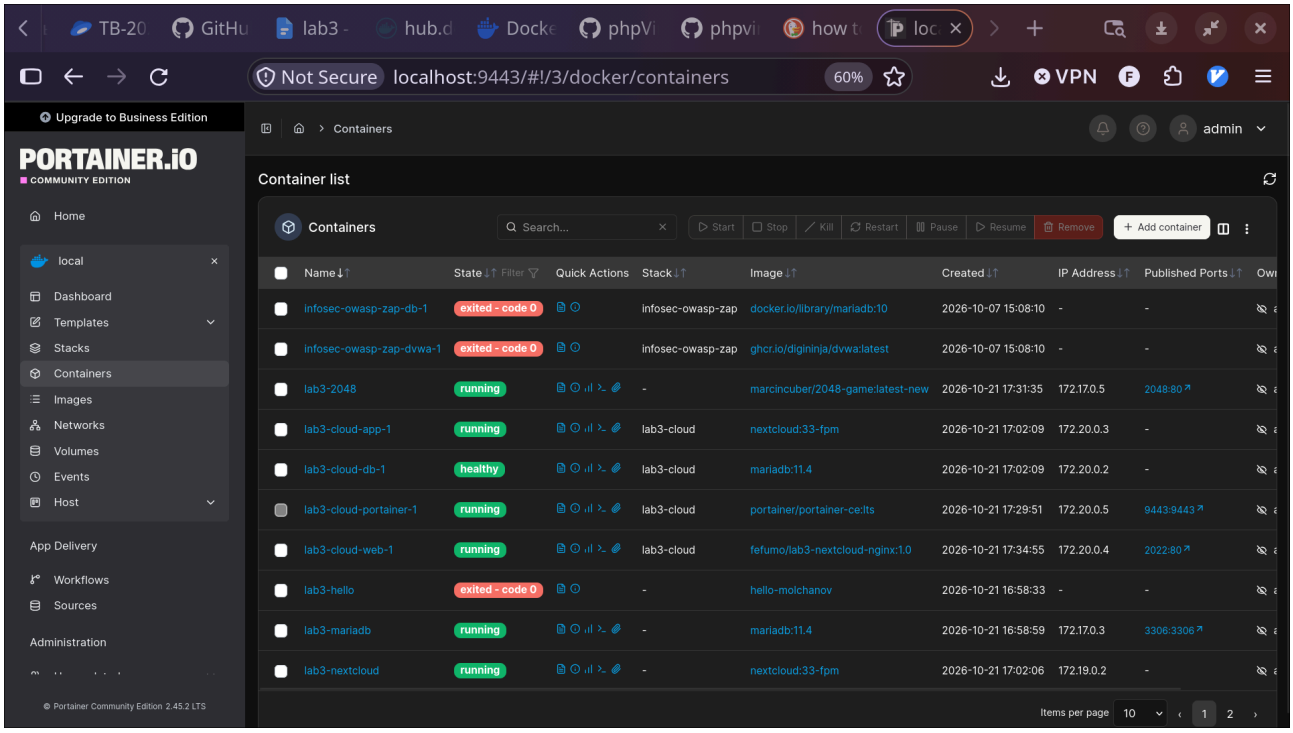


Рис. 2. Portainer: контейнеры локального Docker Engine.

Portainer позволяет просматривать состояние контейнеров, журналы и статистику использования CPU, памяти и сети; запускать, останавливать и перезапускать контейнеры; управлять образами, сетями, томами и стеками приложений. Преимущества — единый веб-интерфейс, наглядное состояние сервисов и выполнение типовых административных операций без ручного ввода команд Docker.

Вывод

Выполнены пункты 1–12: запущены контейнеры приложений, настроены каталоги и порты, создан проект Compose с Nginx, Nextcloud, MariaDB и Portainer. Образ веб-сервера опубликован в Docker Hub. Подтверждены доступность приложений, подключение phpVirtualBox к хосту и просмотр контейнеров в Portainer.